HeartCompass Project Charter

Course Project Charter for the HeartCompass (Digital Immortality) v1.2.2 platform

Project Name	HeartCompass (Digital Immortality) v1.2.2
Project Period	March 30, 2026 to June 14, 2026
Project Type	AI software course project
Team Leader	Tian Bu
Prepared By	Leishiyu Chen, Haojie Hu, Tian Bu, Yihan Liu
Approved By	Tian Bu
Estimated Workload	624 hours
Document Date	May 9, 2026

Project Overview

HeartCompass (Digital Immortality) is planned as a persona-centric AI platform that will transform fragmented interpersonal materials into structured figure profiles and enable context-grounded dialogue. The current main branch has evolved into a v1.2.2 CLI-and-Lark delivery shape with service-backed graph workflows, explicit FRBuildingGraph concurrency control, and Harness documentation assets. The course project will deliver a persona building pipeline (FRBuildingGraph), a conversation pipeline (ConversationGraph), a CLI management tool, and a Lark Bot integration within a single integrated schedule.

Primary Goal

The project will build a platform that collects user and figure-relation data through service-layer contracts, organizes persona information into dimension-based FineGrainedFeed entries with vector embeddings, and uses LangGraph-based pipelines to support both persona construction and sustained dialogue.

Objectives

The project will pursue four closely related objectives. First, it will collect and structure user and figure-relation data behind service-layer contracts so that graph, channel, and CLI modules share a dependable foundation. Second, it will transform raw interpersonal materials into structured FineGrainedFeed entries organized by dimension (personality, interaction style, procedural info, memory) and store them with vector embeddings for semantic recall. Third, it will build a conversation pipeline (ConversationGraph) that generates context-grounded dialogue by combining persona fields, recalled feeds, and short-term memory management. Fourth, it will deliver the system through a CLI-first architecture, Lark Bot integration, and reusable Harness documentation assets, enabling users and maintainers to manage figures, build personas, and sustain dialogue.

Success Criteria

The project will be considered successful if it can organize fragmented interpersonal materials into stable FigureAndRelation entities with structured FineGrainedFeed data, produce persona-building reports that accurately reflect source materials, and support a conversation pipeline whose replies remain consistent with stored persona and recalled memory. Success will also require the CLI and Lark Bot to demonstrate the complete workflow from persona construction to sustained dialogue, including logs access, busy-state feedback, and service-only data access in the main user path, within the planned schedule and estimated workload.

Scope Anchors

The planned system scope will cover the public capabilities exposed through immortality doctor, immortality setup, immortality auth login/logout/whoami, immortality fr list/show/create, immortality logs, immortality lark-service start, FRBuildingGraph (persona construction pipeline), ConversationGraph (dialogue generation pipeline), Lark Bot commands (/menu, /list_available_persons, /show_persona, /build_persona). At the business level, the platform will rely on the core entities User, FigureAndRelation, OriginalSource, FineGrainedFeed, FineGrainedFeedConflict, FROverallUpdateLog, FRBuildingGraphReport, Knowledge, Analysis. These interfaces and entities define the functional boundary of the course project and will be treated as the main scope anchors in later planning, review, and acceptance.

Assumptions and Risks

The charter assumes that users will provide enough narrative or documentary input for the FRBuildingGraph to construct meaningful persona data, and it also assumes that dimension-based FineGrainedFeed organization and pgvector semantic recall will improve dialogue grounding quality. The main risks lie in persona extraction accuracy when source materials are ambiguous, response latency caused by vector recall and LLM inference in the same request chain, conflict accumulation when contradictory information enters the system, single-instance FRBuildingGraph throughput limits, future shared-database and multi-environment configuration complexity, and the sensitivity of interpersonal data, which makes ownership validation and permission control especially important throughout the project.

The document is intentionally written as a pre-execution project plan, even though the current repository already contains working implementations. All schedule language below therefore uses forward-looking, charter-style wording.

RBS

The Responsibility Breakdown Structure (RBS) defines the major responsibility areas of the HeartCompass (Digital Immortality) course project using a numbered hierarchy. The top level identifies three responsibility groups: R1 (Foundation and Persona Input), R2 (Dialogue and Channel), and R3 (CLI, Harness, and Delivery). Each group is decomposed into responsibility streams (e.g., R1.1, R1.2), which are further divided into specific sub-areas (e.g., R1.1.1, R1.1.2). This numbering ensures clear traceability between responsibility definitions and the work packages in the WBS.

R1.1 – User and Figure-Relation Data Foundation

This responsibility stream will establish the identity and relationship baseline of the platform. It covers two sub-areas: (R1.1.1) user identity and authentication, including registration, login, and profile management with MBTI collection; and (R1.1.2) FigureAndRelation entity management, including the

creation, update, and querying of figure-relation records with FigureRole assignment (SELF, FAMILY, FRIEND, MENTOR, COLLEAGUE, PARTNER, PUBLIC_FIGURE, STRANGER) and ownership validation. All downstream modules depend on the data integrity guaranteed by this stream.

R1.2 – Persona Building and Knowledge Management

This stream will convert fragmented interpersonal materials into structured persona data. It covers two sub-areas: (R1.2.1) the FRBuildingGraph pipeline, which preprocesses raw input, extracts FineGrainedFeed entries across four dimensions (PERSONALITY, INTERACTION_STYLE, PROCEDURAL_INFO, MEMORY), and updates FigureAndRelation core fields; and (R1.2.2) conflict detection, update logging, and embedding generation, including FineGrainedFeedConflict tracking, FROverallUpdateLog recording, and pgvector-based vector storage for semantic recall. The main responsibility is to ensure that stored persona data remains retrievable, well-typed, and suitable for downstream dialogue and recall.

R2.1 – Conversation and Dialogue System

This responsibility stream will organize the dialogue core of HeartCompass. It covers two sub-areas: (R2.1.1) the ConversationGraph pipeline, which loads persona and recalled feeds, assembles system prompts with persona markdown, and generates context-grounded replies; and (R2.1.2) short-term memory management, including round-based message trimming, rolling conversation summary, service-backed persona loading, and feed synchronization back to FigureAndRelation core fields to maintain coherence over extended dialogue sessions.

R2.2 – Channel Integration and Lark Bot

This stream will be responsible for the channel integration mainline. It covers two sub-areas: (R2.2.1) Lark WebSocket connection and message handling, including event reception, user-to-FR mapping via open_id, and timed message batching that groups incoming messages before dispatching to ConversationGraph; and (R2.2.2) bot command menu and card-based output, including persona viewing, persona building triggers, FR switching commands, structured card templates for delivering reports and conversation replies, and busy-state feedback when FRBuildingGraph is already running.

R3.1 – CLI, Harness, Packaging, and Deployment

This stream will provide the management interface and prepare the system for delivery. It covers two sub-areas: (R3.1.1) CLI framework development, including the immortality command tree (doctor, setup, auth, fr, logs, lark-service), JSON output mode, and local session management under ~/.immortality/; and (R3.1.2) system packaging, Harness documentation, and deployment, including Docker Compose automation for PostgreSQL, database migration via Alembic, PyPI publishing as digital-immortality, regression testing, and reusable skill assets for course delivery.

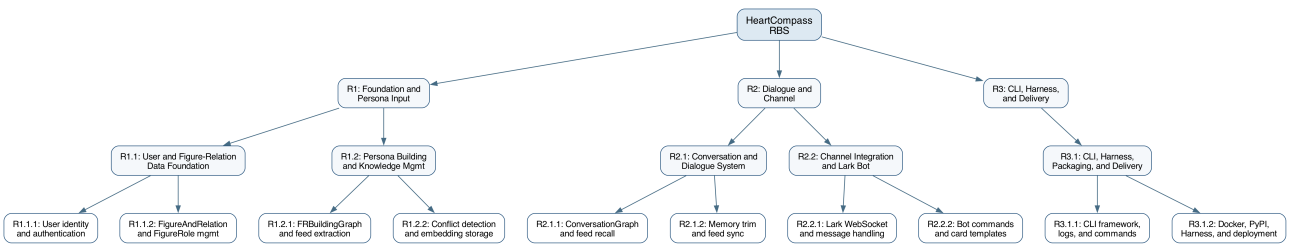

Figure 1. Responsibility Breakdown Structure with numbered hierarchy (R1-R3).

WBS

The Work Breakdown Structure (WBS) translates the RBS responsibility areas into five numbered work packages (WP1–WP5), each containing individually tracked sub-tasks. The project estimates a total workload of 624 hours distributed as follows.

The workload estimate places the greatest emphasis on the persona building and conversation pipelines while reserving substantial effort for data foundation, service alignment, channel integration, CLI delivery, and Harness documentation. This distribution is intended to keep the project academically credible as a software management exercise while still reflecting the real technical center of gravity of HeartCompass.

The following summary table shows the effort distribution across the five major work packages. A detailed task schedule with sub-task-level assignments follows.

Work Package	RBS Ref	Hours
WP1: User and Figure-Relation Data Foundation	R1.1	84
WP2: Persona Building and Knowledge Management	R1.2	120
WP3: Conversation and Dialogue System	R2.1	132
WP4: Channel Integration and Lark Bot	R2.2	126
WP5: CLI, Harness, Packaging, and Deployment	R3.1	162
Total		**624**

RBS-to-WBS Traceability

The WBS is derived directly from the RBS. Each Level-2 RBS responsibility area (R1.1, R1.2, R2.1, R2.2, R3.1) maps one-to-one to a work package (WP1–WP5). Within each work package, the numbered sub-tasks correspond to the Level-3 RBS sub-areas. For example, WP1.1 and WP1.2 trace to R1.1.1 (user identity), while WP1.3 traces to R1.1.2 (FigureAndRelation management), and WP5.6 traces to R3.1.2 (delivery plus Harness documentation). This structure ensures that completing all sub-tasks within a work package fulfills the corresponding RBS responsibility, and that no responsibility area is left without concrete, trackable deliverables.

Each work package is derived from its corresponding RBS responsibility area and is further decomposed into numbered sub-tasks. Every sub-task carries an explicit hour estimate, a week assignment, and an RBS traceability reference, so that progress can be tracked quantitatively at any point during execution. Review and test activities are embedded within each work package rather than treated as separate phases, ensuring that verification happens close to the implementation it validates.

Detailed Task Schedule

Task ID	Task Name	RBS	Hours	Weeks
WP1.1	User registration and authentication	R1.1	20	W3
WP1.2	Profile management and MBTI collection	R1.1	22	W3
WP1.3	FigureAndRelation CRUD and FigureRole	R1.1	24	W4
WP1.4	Data foundation review and test	R1.1	18	W4

WP2.1	FRBuildingGraph preprocessing pipeline	R1.2	28	W4–W5
WP2.2	Fine-grained feed extraction and storage	R1.2	26	W5
WP2.3	Conflict detection and update logging	R1.2	22	W5–W6
WP2.4	Embedding generation and vector recall	R1.2	26	W6
WP2.5	Persona building review and test	R1.2	18	W6
WP3.1	Service-backed ConversationGraph recall	R2.1	28	W5–W6
WP3.2	Short-term memory trim and summary	R2.1	30	W6–W7
WP3.3	LLM call and response generation	R2.1	28	W7
WP3.4	Feed sync and prompt alignment	R2.1	26	W7–W8
WP3.5	Conversation system review and test	R2.1	20	W8
WP4.1	Lark WebSocket and message handling	R2.2	30	W6–W7
WP4.2	Bot menu and persona build commands	R2.2	26	W7
WP4.3	Message batching and card templates	R2.2	24	W7–W8
WP4.4	Service-only data access alignment	R2.2	26	W8
WP4.5	Busy-state cards and integration test	R2.2	20	W8
WP5.1	CLI framework, doctor/setup, and logs	R3.1	32	W7–W8
WP5.2	CLI auth, FR management, and JSON contracts	R3.1	30	W8–W9
WP5.3	Docker support and database automation	R3.1	30	W9
WP5.4	Graph/channel/CLI regression testing	R3.1	30	W9–W10
WP5.5	PyPI packaging and publishing	R3.1	22	W10
WP5.6	Final docs, Harness assets, and deployment	R3.1	18	W11
			624	

WP1: User and Figure-Relation Data Foundation [R1.1] – 84 hours

This work package builds the minimum data layer required by every other module. It contains four sub-tasks: WP1.1 implements user registration and authentication with session management (20h, W3); WP1.2 adds profile management with MBTI collection and open_id binding (22h, W3); WP1.3 delivers FigureAndRelation CRUD with FigureRole assignment and ownership validation (24h, W4); and WP1.4 conducts the review and test cycle covering CRUD reliability, permission isolation, and role-based data consistency (18h, W4).

WP2: Persona Building and Knowledge Management [R1.2] – 120 hours

This work package handles the transformation of raw interpersonal materials into structured persona data. WP2.1 implements the FRBuildingGraph preprocessing pipeline, including source validation, content cleaning, and OriginalSource persistence (28h, W4–W5); WP2.2 adds FineGrainedFeed extraction across four dimensions (PERSONALITY, INTERACTION_STYLE, PROCEDURAL_INFO, MEMORY) with upsert logic (26h, W5); WP2.3 builds conflict detection, FineGrainedFeedConflict tracking, and FROverallUpdateLog recording (22h, W5–W6); WP2.4 delivers embedding generation via pgvector and semantic vector recall (26h, W6); and WP2.5 runs the review and test cycle for extraction accuracy and retrieval behavior (18h, W6).

WP3: Conversation and Dialogue System [R2.1] – 132 hours

This work package constructs the dialogue workflow. WP3.1 builds the service-backed ConversationGraph persona loading and feed recall pipeline, including persona markdown assembly and MEMORY/PROCEDURAL_INFO semantic retrieval (28h, W5–W6); WP3.2 implements short-term memory trimming with round-based cutoff and rolling conversation summary generation (30h, W6–W7); WP3.3 implements LLM call and response generation with system prompt assembly (28h, W7); WP3.4 adds feed synchronization back to FigureAndRelation core fields and prompt alignment for current main-branch behavior (26h, W7–W8); and WP3.5 conducts the review and test cycle for persona consistency and dialogue coherence (20h, W8).

WP4: Channel Integration and Lark Bot [R2.2] – 126 hours

This work package implements the channel integration mainline. WP4.1 builds Lark WebSocket connection and message handling with open_id user mapping (30h, W6–W7); WP4.2 adds bot menu commands for persona viewing, building, and FR switching (26h, W7); WP4.3 implements timed message batching and structured card templates for reports and replies (24h, W7–W8); WP4.4 delivers service-only data access alignment between channel, graph, and service layers (26h, W8); and WP4.5 runs the review and test cycle for end-to-end message flow, command reliability, and busy-state user feedback (20h, W8).

WP5: CLI, Harness, Packaging, and Deployment [R3.1] – 162 hours

This work package completes the product and prepares it for delivery. WP5.1 develops the CLI framework with doctor, setup, and logs commands, including Docker-based PostgreSQL provisioning and database initialization (32h, W7–W8); WP5.2 builds CLI auth, FR management, and JSON output contracts (30h, W8–W9); WP5.3 implements Docker Compose support and database automation via Alembic migrations (30h, W9); WP5.4 conducts graph, channel, and CLI regression testing across all pipelines (30h, W9–W10); WP5.5 handles PyPI packaging and publishing as digital-immortality (22h, W10); and WP5.6 delivers final documentation, Harness skill assets, and deployment (18h, W11).

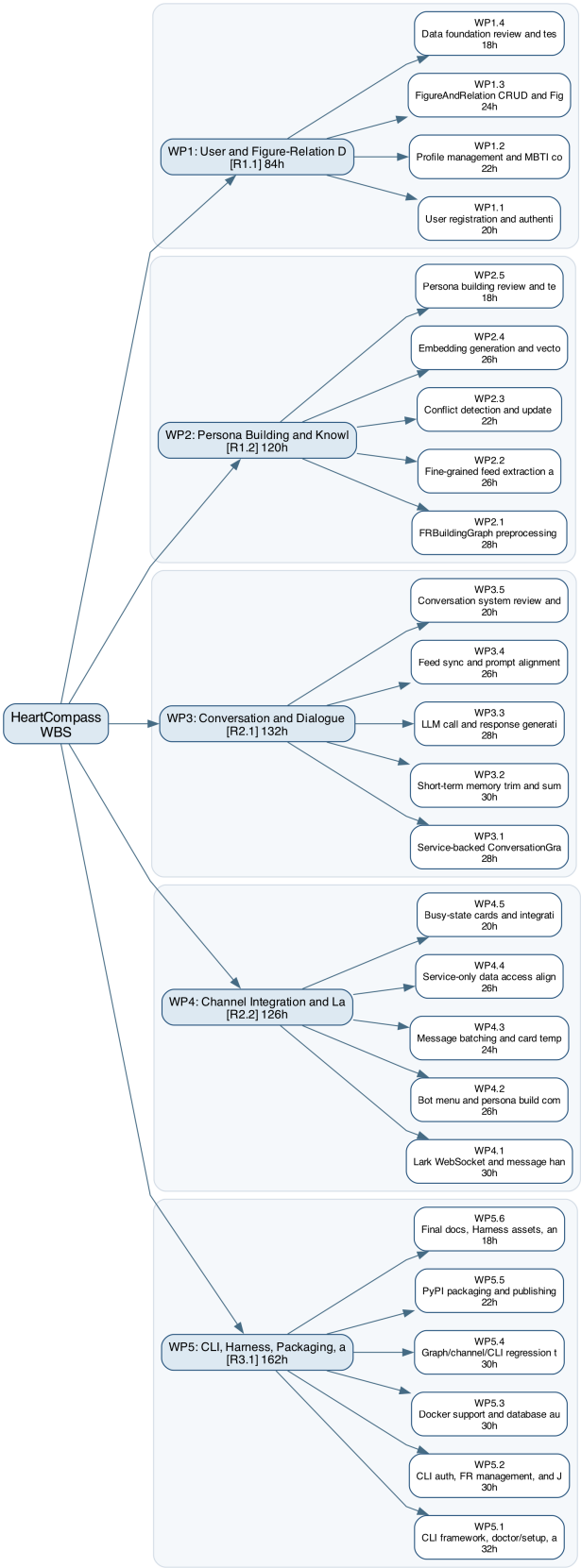

Figure 2. Work Breakdown Structure with numbered sub-tasks and RBS references.

Gantt Chart

The project schedule is organized at the sub-task level across eleven weeks. Each numbered task (WP1.1 through WP5.6) appears as an individual bar on the Gantt chart, grouped under its parent work package. This level of detail allows the team to verify specific task completion at any weekly checkpoint and to identify schedule slippage before it compounds.

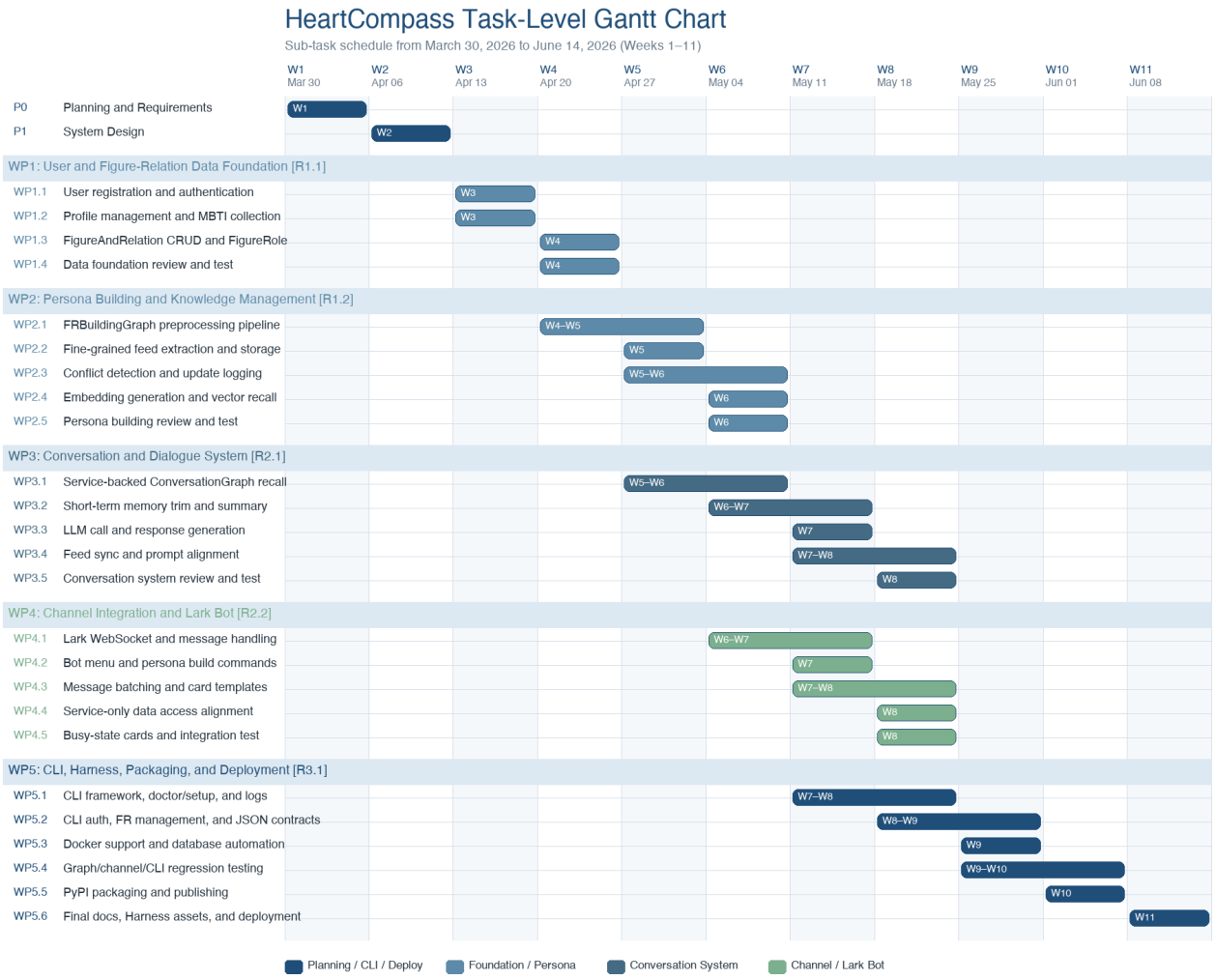

Figure 3. Task-level Gantt chart for HeartCompass (Weeks 1–11).

The schedule follows a deliberate progression: planning and design occupy Weeks 1–2, data foundation is built in Weeks 3–4, persona building and conversation pipelines develop in Weeks 4–8, channel integration with Lark Bot and service alignment progresses in parallel during Weeks 6–8, CLI, Harness, and packaging runs in Weeks 7–9, and system testing through final deployment closes out Weeks 9–11. Each work package includes its own review and test sub-task, so quality verification is distributed throughout the schedule rather than concentrated at the end.

Weekly Milestone Summary

The weekly milestone table below summarizes the expected task completions for each week. At the end of any given week, the team should be able to report which task IDs have been completed, enabling quantitative progress tracking against the baseline schedule.

Week	Dates	Tasks Completing
W1	Mar 30 – Apr 05	P0
W2	Apr 06 – Apr 12	P1

W3	Apr 13 – Apr 19	WP1.1, WP1.2
W4	Apr 20 – Apr 26	WP1.3, WP1.4
W5	Apr 27 – May 03	WP2.1, WP2.2
W6	May 04 – May 10	WP2.3, WP2.4, WP2.5, WP3.1
W7	May 11 – May 17	WP3.2, WP3.3, WP4.1, WP4.2
W8	May 18 – May 24	WP3.4, WP3.5, WP4.3, WP4.4, WP4.5, WP5.1
W9	May 25 – May 31	WP5.2, WP5.3
W10	Jun 01 – Jun 07	WP5.4, WP5.5
W11	Jun 08 – Jun 14	WP5.6

Critical Path Diagram

The critical path identifies the longest dependency chain that determines whether the project can finish on time. For HeartCompass, the critical path runs through: Planning (W1) → System Design (W2) → WP1 Data Foundation (W3–W4) → WP2 Persona Building (W4–W6) → WP4 Channel Integration and Service Alignment (W6–W8) → WP5 CLI, Harness, and Packaging (W7–W9) → WP5.4 System Testing (W9–W10) → WP5.5 PyPI Publishing (W10) → WP5.6 Documentation and Deployment (W11). The conversation and dialogue system (WP3) develops in parallel and converges during system integration.

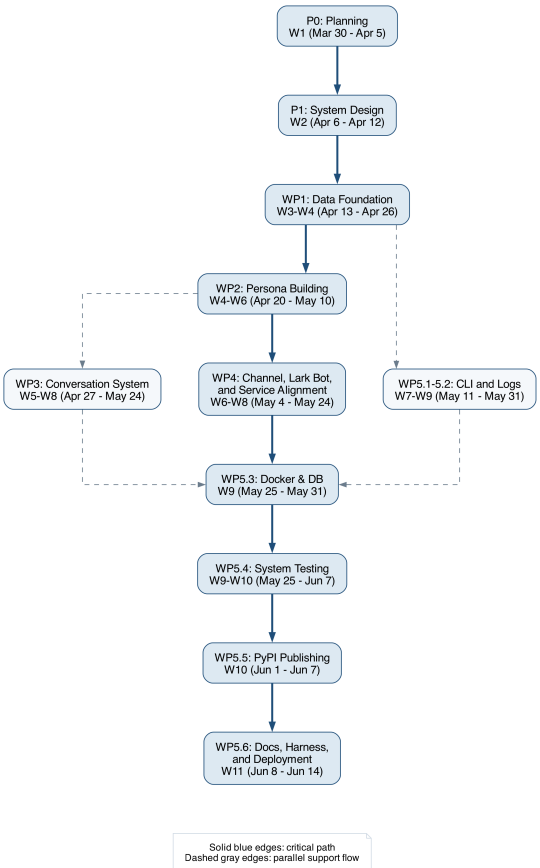

Figure 4. Critical path diagram with work package and task references.

Critical Path Interpretation

The project will begin with requirements gathering and system design because neither the data model nor the downstream AI modules can be stabilized before those decisions are made. Once the FigureAndRelation model and persona building pipeline are defined, the channel integration mainline becomes the controlling branch because it depends on persona construction, dialogue generation, and Lark Bot integration in sequence. The conversation and dialogue system remains a major parallel branch; however, its outputs converge with the CLI and packaging flow during system integration rather than controlling the overall finish date. This dependency structure makes system testing (WP5.4), PyPI publishing (WP5.5), and final deployment (WP5.6) the final schedule gates.

Planned Deliverables

By the end of the planned schedule, the team will deliver a documented data model covering FigureAndRelation and FineGrainedFeed entities, a persona building pipeline (FRBuildingGraph) with conflict detection, update logging, and explicit single-run concurrency control, a conversation pipeline (ConversationGraph) with short-term memory management, a Lark Bot integration for daily interaction, a CLI tool (immortality) for system management and log inspection, a PyPI-publishable package (digital-immortality), and a final integration package that includes testing evidence, Harness documentation, and Docker-based deployment.

Project Governance Notes

Progress will be reviewed at the end of each week against the task schedule. The team will track completed task IDs (e.g., WP1.1, WP2.3) and compare them against the planned weekly milestones. Any scope adjustment will be evaluated against its likely effect on timeline, integration complexity, service contract stability, and delivery quality before it is accepted into the project plan. Reusable documentation and review skills will be treated as governance assets rather than one-off notes.

Testing and Delivery Criteria

The final submission will need to satisfy several quality conditions at the same time. The charter must remain fully in English, use consistent future tense, and keep all dates aligned to 2026. Its sections, charts, workload estimate, and narrative explanations must remain mutually consistent, and the final PDF must be printable, readable, and professionally formatted without unstable spacing, clipped graphics, or broken page flow.